



**COSTA RICA INSTITUTE OF TECHNOLOGY  
ALAJUELA ACADEMIC CENTER  
COMPUTER ENGINEERING DEPARTMENT**

# **Project 0: The Narrow Bridge Simulation**

*An Implementation of Concurrency and Process Synchronization*

**IC-6600 Principles of Operating Systems  
I Semester, 2026**

By:

**Angie Esquivel López  
Alejandro Quesada Calderón  
Esteban Espinoza Solano**

**Prof. Eddy Ramírez Jiménez**

Alajuela, March 23, 2026

# Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                | <b>3</b>  |
| <b>2</b> | <b>Theoretical Framework</b>                                       | <b>4</b>  |
| 2.1      | Operating Systems and Interfaces                                   | 4         |
| 2.2      | Concurrency and Threads                                            | 4         |
| 2.3      | CPU Scheduling                                                     | 5         |
| 2.4      | Process Synchronization and Deadlocks                              | 5         |
| <b>3</b> | <b>Solution Description</b>                                        | <b>5</b>  |
| 3.1      | Overview                                                           | 5         |
| 3.2      | Module Structure                                                   | 6         |
| 3.3      | Data Structures                                                    | 6         |
| 3.3.1    | Configuration ( <code>Config</code> )                              | 6         |
| 3.3.2    | FIFO Wait Queue ( <code>FifoQueue</code> / <code>FifoNode</code> ) | 7         |
| 3.3.3    | Bridge State ( <code>Bridge</code> )                               | 7         |
| 3.3.4    | Traffic Light State ( <code>LightState</code> )                    | 8         |
| 3.3.5    | Officer Sides ( <code>OfficerState</code> )                        | 8         |
| 3.4      | Synchronization Design                                             | 8         |
| 3.4.1    | The Entry Predicate                                                | 8         |
| 3.4.2    | Per-Vehicle Condition Variables                                    | 9         |
| 3.4.3    | Wake Protocol                                                      | 10        |
| 3.4.4    | Physical Bridge Traversal                                          | 11        |
| 3.5      | Operating Modes                                                    | 11        |
| 3.5.1    | Carnage Mode                                                       | 11        |
| 3.5.2    | Semaphore Mode                                                     | 11        |
| 3.5.3    | Officer Mode                                                       | 11        |
| 3.6      | Traffic Generation                                                 | 12        |
| 3.7      | Build System                                                       | 12        |
| 3.8      | Graphical Interface                                                | 13        |
| 3.8.1    | Structured Log Protocol                                            | 13        |
| 3.8.2    | SDL2 and Rendering                                                 | 13        |
| 3.9      | Deadlock and Starvation Prevention                                 | 14        |
| <b>4</b> | <b>Conclusions</b>                                                 | <b>14</b> |
| 4.1      | Concurrency and deadlock prevention                                | 14        |
| 4.2      | Performance of the modes                                           | 14        |
| 4.3      | Impact of randomness on the simulation                             | 15        |
| <b>5</b> | <b>Description of the Experience</b>                               | <b>15</b> |
| <b>6</b> | <b>Personal Key Takeaways</b>                                      | <b>16</b> |
|          | <b>References</b>                                                  | <b>16</b> |

# 1 Introduction

The development of modern computing relies on concurrent execution and efficient resource management. This project aims to provide a solid understanding of both the basic theory and practice of these systems. The primary purpose of this project is to build with basic thread management and concurrency, and with that solving classic operating system problems. This implementation exposes the developer to an environment with multiple processes that must be scheduled by the operating system.

To achieve this, the project simulates the narrow bridge problem. The simulation models a city bridge connecting an East and West sector. Because the bridge is obsolete and only wide enough for one car, it only allows unidirectional traffic at any given time. This physical constraint demands careful administration to maximize the utilization of the bridge, minimize waiting times, and ensure fair allocation of the resource.

The solution is programmed entirely in C on a Linux environment. It utilizes the Pthreads library to handle the execution threads. Because Pthreads is based on the POSIX standard, it serves as a highly useful tool for utilizing system services through system calls. Every entity in the simulation, such as vehicles, traffic lights, and police officers, is represented by an individual thread. Consequently, the project requires strict administration of process synchronization to maintain order and actively prevent deadlocks.

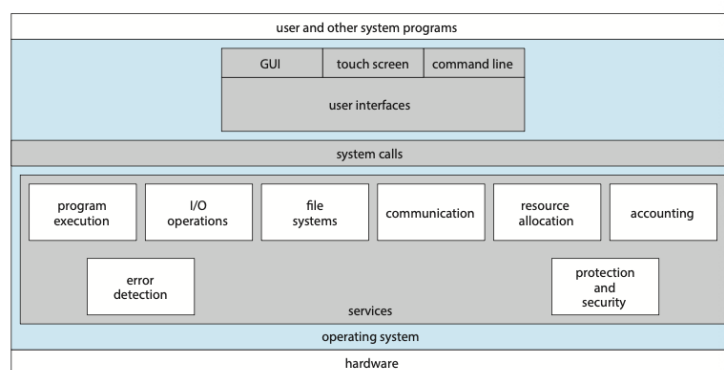
## 2 Theoretical Framework

This section outlines the foundational concepts underpinning the concurrent simulation, establishing the theoretical basis for managing shared resources in a multiprogrammed environment.

### 2.1 Operating Systems and Interfaces

The foundation of any hardware-level simulation is the operating system itself. An operating system is software that manages a computer’s hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware (Silberschatz, Galvin, & Gagne, 2018, Chap. 1, p. 3).

To interact with this underlying system, programs must use specific interfaces. System calls provide the essential interface between a running user program and the operating system kernel. They allow processes to request privileged operations safely, such as creating new threads, managing hardware resources, or performing I/O operations (Silberschatz et al., 2018, Chap. 2, p. 62).



### 2.2 Concurrency and Threads

When multiple operations are initiated via system calls, the system must handle them simultaneously. We use this conceptual term to refer to a host of problems that arise, and must be addressed, when working on many things at once (i.e., concurrently) in the same program. The problems of concurrency arose first within the operating system itself; the OS is juggling many things at once, first running one process, then another, and so forth (Arpaci-Dusseau & Arpaci-Dusseau, 2008, Chap. 2, p. 6).

To achieve this concurrency efficiently, modern operating systems utilize threads. A thread is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals. A traditional (or heavyweight) process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time (Silberschatz et al., 2018, Chap. 4, p. 160).

## 2.3 CPU Scheduling

Because concurrent execution relies on sharing a limited number of processors, dynamic administration becomes critical. A process migrates among the ready queue and various wait queues throughout its lifetime. The role of the CPU scheduler is to select from among the processes that are in the ready queue and allocate a CPU core to one of them. The CPU scheduler must select a new process for the CPU frequently. An I/O-bound process may execute for only a few milliseconds before waiting for an I/O request. Although a CPU-bound process will require a CPU core for longer durations, the scheduler is unlikely to grant the core to a process for an extended period. Instead, it is likely designed to forcibly remove the CPU from a process and schedule another process to run (Silberschatz et al., 2018, Chap. 3, p. 113).

## 2.4 Process Synchronization and Deadlocks

When multiple threads execute concurrently and share resources, such as a narrow bridge, data integrity must be maintained. This requires process synchronization. A critical section is a piece of code that accesses a shared resource, usually a variable or data structure. To avoid race conditions, we need to ensure that only one thread can execute within the critical section at a time. This is known as mutual exclusion (Arpaci-Dusseau & Arpaci-Dusseau, 2008, Chap. 28, p. 3).

Improper scheduling and resource allocation among concurrent processes can lead to catastrophic system states. For many applications, a process needs exclusive access to not one resource, but several. Suppose, for example, two processes each want to scan an object with a 3D scanner and then print the front, top, and side view of the object on a printer. Process A requests permission to use the 3D scanner and is granted it. Process B is programmed differently and requests the printer first and is also granted it. Now A asks for the printer, but the request is suspended until B releases it. Unfortunately, instead of releasing the printer, B asks for the 3D scanner. At this point both processes are blocked and will remain so forever. This situation is called a **deadlock** (S.Tanenbaum & Bos, 2023, Chap. 6, p. 437).

# 3 Solution Description

## 3.1 Overview

The narrow bridge simulation is implemented as a multi-threaded C program using the POSIX Threads (Pthreads) library. Every entity in the problem is represented by an independent thread: each vehicle is a thread, each traffic generator is a thread, and the semaphore controller (in **SEMAPHORE** mode) is a thread. A separate process handles the graphical interface by reading the simulation's standard output through a Unix pipe. The build system is managed with **make**, and the graphical visualiser is built with the SDL2 library.

## 3.2 Module Structure

The project is organized into distinct source modules, each with a well-defined responsibility. Source files are placed under `src/`, header files under `include/`, and object files are emitted into `build/` by the build system. The modules are summarized in Table 1.

Table 1: Source modules and their responsibilities.

| Module                         | Responsibility                                                                                                                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>config</code>            | Parses the <code>.config</code> file into a <code>Config</code> struct. Validates all parameters and aborts with a clear error message on invalid input.                                     |
| <code>bridge</code>            | Owns all shared bridge state and every synchronization primitive. Implements the three entry predicates (Carnage, Semaphore, Officer) through a single <code>can_head_enter</code> function. |
| <code>vehicle</code>           | Defines the <code>Vehicle</code> struct and the <code>vehicle_thread</code> routine. Emits structured [SLOT] log lines on every meter advance for the GUI.                                   |
| <code>traffic_generator</code> | Spawns vehicle threads according to an exponential inter-arrival distribution, independently for each side of the bridge.                                                                    |
| <code>semaphore_ctrl</code>    | Implements the traffic-light controller thread (active only in <code>SEMAPHORE</code> mode). Cycles green phases using wall-clock timers read from the configuration.                        |
| <code>officer_ctrl</code>      | Implements the officer controller threads (active only in <code>OFFICER</code> mode). Cycles between sides based using the corresponding $k$ value                                           |
| <code>main</code>              | Entry point. Loads configuration, creates the bridge, conditionally starts the semaphore controller, and joins all threads before teardown.                                                  |
| <code>gui</code>               | A standalone process that reads the simulation’s standard output through a pipe, parses structured log lines, and renders a live SDL2 window.                                                |

## 3.3 Data Structures

### 3.3.1 Configuration (Config)

All simulation parameters are stored in a `Config` struct defined in `config.h`. Each side of the bridge has its own `SideConfig`, containing the exponential arrival mean, speed range,

ambulance percentage, green-light duration, and officer  $K$  value. The bridge length and simulation duration are global parameters. The operating mode is an enumeration with three values: `MODE_CARNAGE`, `MODE_SEMAPHORE`, and `MODE_OFFICER`.

### 3.3.2 FIFO Wait Queue (FifoQueue / FifoNode)

Each side of the bridge maintains a singly-linked FIFO queue of vehicles currently waiting to enter. A `FifoNode` is stack-allocated inside `bridge_enter` for the duration of each vehicle's wait, avoiding heap allocation per vehicle:

```

1 typedef struct FifoNode {
2     pthread_cond_t    cv;           /* this vehicle sleeps here */
3     int               is_ambulance; /* inspected by the bridge */
4     int               id;           /* for logging */
5     struct FifoNode *next;
6 } FifoNode;
7
8 typedef struct {
9     FifoNode *head; /* next vehicle to enter the bridge */
10    FifoNode *tail; /* most recently arrived vehicle */
11    int       size;
12 } FifoQueue;

```

Listing 1: FIFO node and queue structures (`bridge.h`).

Vehicles join the tail in strict arrival order; there is no reordering at any point. Ambulance priority is enforced by inspecting `head->is_ambulance` at the point of entry, not by repositioning nodes. This means every vehicle waits its fair turn in the queue, and only once it reaches the head does its ambulance status influence whether it can proceed onto the bridge.

### 3.3.3 Bridge State (Bridge)

The central shared structure is `Bridge`:

```

1 struct Bridge {
2     int             length;
3     pthread_mutex_t lock;
4     pthread_mutex_t *slots; /* per-meter mutex array */
5
6     Mode            mode;
7
8     FifoQueue        queue[2]; /* one FIFO per side */
9     LightState        light[2]; /* SEMAPHORE mode only */
10
11    int              cars_on_bridge;
12    Direction         current_direction;
13
14    int              waiting[2];

```

```

15     int            ambulances_waiting[2];
16     int            passed_count[2];
17
18     int            current_k_value;
19     int            k_on_bridge;
20     int            ambulance_reset;
21     OfficerState    officer_side[2];
22     Direction       officer_turn;
23     pthread_cond_t  officer_cv;
24     pthread_cont_t  officer_done_cv;
25 };

```

Listing 2: Bridge structure (bridge.h).

The `slots` array is a dynamically allocated array of mutexes, one per meter of bridge length. A vehicle holds `slots[i]` while occupying meter  $i$  and acquires `slots[i+1]` before releasing `slots[i]`, producing sequential movement with no two vehicles sharing a meter. The `light` array is indexed by `Direction` (`EAST=0`, `WEST=1`) and is only meaningful in `SEMAPHORE` mode; in other modes it is set to `LIGHT_OFF`.

### 3.3.4 Traffic Light State (LightState)

The semaphore mode introduces a `LightState` enumeration with three values: `LIGHT_GREEN`, `LIGHT_RED`, and `LIGHT_OFF`. Only one side can be green at a time. The `bridge_set_light` function, called exclusively by the semaphore controller thread, updates both entries in `light[]` under `bridge->lock` and then wakes the head of each queue so they can re-evaluate the entry predicate.

### 3.3.5 Officer Sides (OfficerState)

The officer mode introduces a `OfficerState` enumeration with three values: `SAME`, `OPP`, and `OFFICERS_DAY_OFF`. Only one officer can be awake at the time (letting the cars in). The `bridge_set_officer` function, called exclusively by the officer controller thread, updates both entries in `officer_side[]` under `bridge->lock` and then wakes the head of each queue so they can re-evaluate the entry predicate.

## 3.4 Synchronization Design

### 3.4.1 The Entry Predicate

All synchronization decisions flow through a single static function, `can_head_enter`, which returns 1 if the vehicle at the head of a given side's FIFO is currently permitted to enter the bridge. This is the only place where mode-specific rules diverge:

```

1 static int can_head_enter(const Bridge *b, Direction side)
2 {
3     const FifoNode *head      = b->queue[side].head;
4     const FifoNode *opp_head = b->queue[opp].head;

```



```

5
6     int bridge_ok  = (b->cars_on_bridge == 0 ||
7                       b->current_direction == side);
8     int must_yield = (!head->is_ambulance &&
9                       opp_head != NULL      &&
10                      opp_head->is_ambulance);
11
12     if (b->mode == MODE_SEMAPHORE) {
13         if (head->is_ambulance && b->light[side] == LIGHT_RED)
14             return (b->cars_on_bridge == 0) && !must_yield;
15         if (!head->is_ambulance && b->light[side] == LIGHT_RED)
16             return 0;
17     }
18     if (b->mode == MODE_OFFICER) {
19         OfficerState my_side = b->officer_side[side];
20         if (head->is_ambulance)
21         {
22             if (my_side == OPP)
23                 return bridge_clear && !must_yield;
24             return bridge_ok && !must_yield;
25         }
26         if (my_side == OPP || my_side == OFFICERS_DAY_OFF)
27             return 0;
28         return bridge_ok && !must_yield && (b->current_k_value > 0);
29
30     return bridge_ok && !must_yield;
31 }

```

Listing 3: Entry predicate (simplified from `bridge.c`).

The logic for each combination of vehicle type, light state, and mode is summarized in Table 2.

### 3.4.2 Per-Vehicle Condition Variables

Each `FifoNode` carries its own `pthread_cond_t`, initialized when the vehicle thread calls `bridge_enter` and destroyed before it returns. This means only the specific vehicle that should wake up is signalled — there is no broadcast, no spurious wake-up storm, and no need for an additional ready flag. The wait loop in `bridge_enter` is:

```

1 while (q->head != &node || !can_head_enter(b, side))
2     pthread_cond_wait(&node.cv, &b->lock);

```

Listing 4: Wait loop in `bridge_enter`.

The double condition guards against two distinct cases: the vehicle is not yet at the front of the queue (`q->head != &node`), or it is at the front but the bridge state does not yet permit entry.

Table 2: Entry conditions by vehicle type and mode.

| Mode      | Vehicle                          | Condition to enter                                                                                              |
|-----------|----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Carnage   | Any car                          | Bridge empty or same direction, and no ambulance at the opposite head.                                          |
| Carnage   | Ambulance                        | Bridge empty or same direction (not blocked by opposite ambulance either).                                      |
| Semaphore | Car, green                       | Same as Carnage.                                                                                                |
| Semaphore | Car, red                         | Never — must wait for green.                                                                                    |
| Semaphore | Ambulance, green                 | Same as Carnage.                                                                                                |
| Semaphore | Ambulance, red                   | Only when the bridge is completely empty (no oncoming traffic). Light does not change.                          |
| Officer   | Car, current officer side        | Same as Carnage (until $k$ value runs out.                                                                      |
| Officer   | Car, opposite officer side       | Never — must wait for the officer to change sides.                                                              |
| Officer   | Ambulance, current officer side  | Same as Carnage, but the ambulance can enter even if the $k$ value runs out.                                    |
| Officer   | Ambulance, opposite officer side | Only when the bridge is completely empty (no oncoming traffic). Officer changes sides, resetting the $k$ value. |

### 3.4.3 Wake Protocol

Every function that changes bridge state calls `try_wake_head` for the relevant sides. This helper calls `pthread_cond_signal` on the head’s condition variable if and only if `can_head_enter` returns true, preventing unnecessary wakeups. The sites that trigger wakes are:

- **bridge\_enter**: after a vehicle enters the bridge, the new head of the same queue is woken (pipelining), and the opposite head is woken in case it was waiting for the previous ambulance to clear.
- **bridge\_leave**: when the bridge empties, one side is chosen according to the priority order (opposite ambulance, same ambulance, opposite normal, same normal) and its head is woken.
- **bridge\_set\_light**: the semaphore controller wakes both heads after a light flip; the newly-green head may now enter, and a red-side ambulance may be able to cross if the bridge is empty.
- **bridge\_set\_officer**: as the semaphore controller, the officer controller wakes both heads after a change in sides; the current direction head may now enter, and a opposite-side ambulance may be able to cross if the bridge is empty.
- **Ambulance arrival**: when a new ambulance joins a queue, the opposite head is signalled immediately so it can re-evaluate and yield if required.

### 3.4.4 Physical Bridge Traversal

The bridge is modelled as an array of `length` mutexes, one per meter. A vehicle acquires `slots[0]` upon entering and then performs a hand-over-hand lock sequence: it acquires `slots[i+1]` before releasing `slots[i]` as it advances. This prevents two vehicles occupying the same meter and naturally produces the sequential progression required by the problem specification.

## 3.5 Operating Modes

### 3.5.1 Carnage Mode

In Carnage mode neither the semaphore controller nor any other external thread intervenes. The only rules are: vehicles may enter if the bridge is empty or already flowing in their direction, and a normal car must yield if an ambulance is waiting at the head of the opposite queue. Multiple same-direction vehicles pipeline onto the bridge simultaneously.

### 3.5.2 Semaphore Mode

The semaphore controller (`semaphore_ctrl.c`) runs as a dedicated thread that cycles two independent green-phase timers read from the configuration file: `east_green_time` and `west_green_time`. The timers are purely wall-clock based and are not paused, reset, or otherwise affected by bridge events. When a phase expires, `bridge_set_light` is called under `bridge->lock`, the `light[]` entries are updated atomically, and both queue heads are signalled to re-evaluate. Cars already mid-crossing when the light changes complete their crossing normally; the new red-side head simply waits in its condition variable until the next green phase.

Ambulances are the only vehicles exempt from the red-light rule. An ambulance at the head of a red queue is permitted to enter if and only if `cars_on_bridge == 0`. Crossing on red does not alter the light timer in any way.

### 3.5.3 Officer Mode

The officer controller is planned as a pair of dedicated thread that calls a `bridge_set_officer` function each time the  $K$ -vehicle quota (each side has its own  $K$  value) for the current side is exhausted and there is no more cars on the bridge. For normal cars on the active side, the predicate `can_head_enter` checks that the quota is still positive ( $k > 0$ ) and that the usual Carnage conditions (bridge empty or same direction, no opposite ambulance waiting) hold. Each admitted normal car decrements `current_k_value` and increments `k_on_bridge`. When the quota reaches zero, no further normal cars may enter; the bridge continues to drain the cars that are already crossing.

Ambulances are given priority. If an ambulance waits at the head of the active queue:

- If  $k > 0$  it consumes one quota slot (like a normal car).
- If  $k$  is already exhausted, it is allowed to enter without consuming a quota slot (privilege pass).

If an ambulance is at the head of the *blocked* side, it may cross only when the bridge is completely empty. When such a crossing happens, the *active* officer thread's turn is terminated early: `ambulance_reset` is set and the active officer is woken via `officer_cv` to hand over control to the other side.

As of the current implementation the officer mode scaffolding is present in the `Mode` enumeration but not yet active.

## 3.6 Traffic Generation

Two generator threads run concurrently, one per side, each independently sampling inter-arrival times from an exponential distribution with the configured mean. The formula used is  $t = -\mu \ln(1 - u)$  where  $u$  is a uniform random variable in  $[0, 1)$ . Each generator uses a thread-local seed for `rand_r`, avoiding contention on a global RNG. A global atomic counter protected by a dedicated mutex provides unique vehicle IDs. Generators run for `simulation_time` seconds and then wait for all vehicle threads they spawned to finish before exiting, ensuring a clean shutdown without stranded threads.

## 3.7 Build System

The project uses a GNU Makefile with the following design decisions:

- **Source discovery via wildcard:** the line `SRC = $(wildcard $(SRC_DIR)/*.c)` picks up all `.c` files in `src/` automatically. Adding a new module such as `officer_ctrl.c` requires no Makefile change.
- **Separate build directory:** all `.o` files are emitted to `build/`, keeping the source tree clean.
- **Strict compiler flags:** `-Wall -Wextra -Werror` ensure that every warning is treated as a build failure. The simulation binary links with `-pthread` and `-lm`.
- **GUI binary separated:** the graphical interface (`gui.c`) is compiled as a distinct target, `bridge_gui`, because it requires `-lSDL2` which the simulation does not need. The `run-gui` target pipes the simulation into the GUI process: `./bridge_sim $(CONFIG) | ./bridge_gui`.
- **Platform detection:** the Makefile detects the operating system and architecture using `uname -s` and `uname -m` at parse time. On Apple Silicon macOS it adds `-I/opt/X11/include` and `-I/opt/homebrew/include`; on Linux these flags are omitted because SDL2 headers are in the standard system paths.
- **SDL2 flag delegation:** the GUI compile flags are obtained via `sd12-config --cflags` and `sd12-config --libs` when available, falling back to `pkg-config` and finally to a bare `-lSDL2`.

## 3.8 Graphical Interface

The GUI is implemented as a second standalone process in `gui.c`. It communicates with the simulation exclusively through standard I/O: the simulation emits structured log lines to `stdout` and the GUI reads them from `stdin` through a Unix pipe. This design means the simulation requires no modification to support the GUI, and the GUI can be dropped or replaced without touching the simulation code.

### 3.8.1 Structured Log Protocol

Three machine-readable line formats are emitted by the simulation alongside the human-readable console output:

- `[SLOT id slot bridge_len direction is_ambulance]` — emitted by the vehicle thread on every meter advance. `slot` is the current 0-based meter index, or `-1` when the vehicle has fully exited the bridge. This allows the GUI to track every vehicle's position with meter-level precision.
- `[QUEUE EAST|WEST total ambulances]` — emitted by `bridge_enter` and `bridge_leave` whenever the waiting counters change, so the GUI always reflects the current queue lengths.
- `[DIRECTION EAST|WEST|NONE]` — emitted by `bridge_set_light` and `bridge_leave` whenever the authorised direction changes.

All other lines (human-readable vehicle and bridge events) are forwarded to a circular log buffer rendered in a panel at the bottom of the window.

### 3.8.2 SDL2 and Rendering

SDL2 was chosen over Xlib because it is the only graphics library that works natively on both supported platforms (x86-64 Linux and Apple Silicon macOS) without requiring XQuartz or any other compatibility layer. On Linux SDL2 is installed via `apt install libsdl2-dev`; on macOS via `brew install sdl2`.

Because SDL2 provides no built-in text rendering without the optional `SDL2_ttf` extension library (which was not available in the build environment), a  $5 \times 7$  pixel bitmap font covering the full printable ASCII range is baked directly into `gui.c` as a 95-entry lookup table of 5 bytes per glyph. Each set bit in a column byte corresponds to one lit pixel row; the font is rendered at  $2 \times$  scale, yielding  $10 \times 14$  pixel glyphs. This eliminates every external font dependency.

The GUI parser runs on a dedicated thread that feeds shared state under a mutex. The main thread runs the SDL2 event loop and redraws at approximately 15 frames per second using `SDL_GetTicks` for frame throttling. High-DPI displays (Retina on macOS) are handled by calling `SDL_GetRendererOutputSize` and `SDL_RenderSetScale` so the logical coordinate system remains fixed regardless of pixel density.

### 3.9 Deadlock and Starvation Prevention

Several design choices explicitly prevent deadlock and starvation:

- **Single global lock:** all bridge state is protected by a single `pthread_mutex_t`. There is never more than one lock held at a time (except the per-meter slot mutex, which is acquired after the global lock is released), so lock-ordering deadlocks cannot occur.
- **No busy-waiting:** all waiting is done through `pthread_cond_wait`, which atomically releases the lock and suspends the thread.
- **FIFO guarantees no starvation within a side:** because vehicles join the queue in arrival order and are never reordered, every vehicle is eventually guaranteed to reach the head.
- **Direction alternation:** when the bridge empties in Carnage mode, the opposite side is given priority if it has waiting vehicles, preventing indefinite starvation of one side by a continuous stream from the other.
- **Ambulance priority bounded:** ambulances do not jump the FIFO queue. Their privilege is expressed only at the head: a non-ambulance at the head of the opposite queue must yield, but it will eventually enter once the ambulance clears the bridge.

## 4 Conclusions

Through the design, implementation, and testing of this project, several technical conclusions regarding concurrency, thread management, and process scheduling were reached. The execution of the program yielded the following key observations:

### 4.1 Concurrency and deadlock prevention

Proper implementation of mutexes and synchronization tools is absolutely essential to maintain data integrity and prevent deadlocks. Results confirmed that strict access control to the bridge was necessary so that cars, traffic officers, and semaphores could execute their routines safely without resorting to busy waiting.

### 4.2 Performance of the modes

The simulation proved that the **Traffic Officer mode** is the best strategy for maximizing bridge utilization and minimizing wait times, overcoming the flaws of the other two modes:

- **Carnage:** Allows continuous flow but causes resource starvation if one direction has heavy traffic.
- **Semaphore:** Guarantees strict fairness but is inefficient, often leaving the bridge without use, if the green-light side has no waiting cars.

- **Traffic Officer:** Uses dynamic scheduling. The K quota prevents starvation, while the “early switch” mechanism (yielding the turn immediately if the active side is empty) maximizes bridge utilization and reduces waiting times.

### 4.3 Impact of randomness on the simulation

Using an exponential distribution for vehicle arrival times and assigning varying speeds to each car accurately simulated real-world unpredictability. This randomness thoroughly stress-tested the system’s logic, proving that the simulation could handle irregular traffic spikes and different crossing durations independently for each side without failing.

## 5 Description of the Experience

Prior to this project, the team shared a common starting point: little to no practical experience with concurrent programming in C. For most members, this language had only been used in the context of simple, self-contained algorithms or short single-file programs, and the `pthread` library was unfamiliar territory. This made the initial design phase particularly challenging, as the project required not only learning a new tool, but also applying it to a problem where correctness depended on getting the concurrency model exactly right.

The first major obstacle the team encountered was grasping the mental model behind threads. Understanding how multiple threads share state, and more importantly how that shared state can become inconsistent without proper synchronization, required a shift in thinking that went beyond what any of us had previously practiced. Concepts such as mutex locks, condition variables, and the `pthread_cond_wait` / `pthread_cond_signal` pattern had to be understood not just in isolation, but in the context of a live system where several threads compete and cooperate simultaneously.

Translating the three bridge management modes (Carnage, Semaphore, and Officer) into thread-safe implementations was where most of the confusion accumulated. Each mode introduced a different set of rules about when a vehicle thread was allowed to proceed, and finding a clean, unified predicate (`can_head_enter`) that correctly encoded those rules for all three modes required multiple iterations and a great deal of debugging. The Officer mode in particular demanded careful design, as it introduced a second layer of coordination between vehicle threads and a dedicated controller thread, with the additional complexity of ambulance priority handling and the K-slot accounting system.

Beyond the technical challenges, the project reinforced the importance of team communication as a practical engineering skill. There were moments where the group’s collective understanding of the problem diverged, leading to implementations that were individually logical but incompatible with each other. Resolving those divergences required the team to slow down, align on a shared model, and only then return to the code.

Version control also played a meaningful role throughout the development process. Managing a shared codebase across multiple contributors, handling branches, resolving conflicts, and maintaining a coherent commit history were skills that the team exercised continuously and that will remain useful well beyond this course.

Overall, the project succeeded in its formative goal: it placed the team in a situation where

the theory of operating systems (scheduling, synchronization, deadlock prevention) had to be confronted directly and implemented responsibly. The experience was demanding, but it left each member with a concrete understanding of concurrent systems that lectures alone could not have provided.

## 6 Personal Key Takeaways

- Alejandro: "I personally did not have a clue about the usage of pthread prior to the development of this project. I had only used C for very simple, short algorithms, most of the time for monolithic/single-file problem solving and other small applications. Apart from learning about **pthread** and its included functions, I also had to dig into some information about SDL2, a graphic development library for C. In tandem, I employed my —yet to be refined more and more— knowledge with git and GitHub, by creating the project repository, handling commit diversions, branches and other common basic advantages of sourcing a github repository."
- Angie: "The most important thing I learned in this project was how to use the **phthreads** library to manage threads and concurrency. Understanding and implementing the different modes (Carnage, Semaphore, and Traffic Officers) was really confusing for my teammates and me at first, especially since we had to represent every entity as an individual thread. However, this experience taught me the value of team communication, which was essential for us to figure out the implementation, resolve our confusion, and complete the project successfully."
- Esteban: "Honestly, I knew nothing about how to use **phthreads** nor the language C. The way to use threads was somewhat confusing to me at the beginning, but after learning a little, I started to understand the conditions, the signals and the parallelization to be more efficient. Nonetheless, the most important thing this experience gave me was that, even if I am not at my best, I have to do the best I can."

## References

- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2008). *Operating systems three easy pieces*. CreateSpace Independent Publishing Platform.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). John Wiley & Sons.
- S.Tanenbaum, A., & Bos, H. (2023). *Modern operating systems* (5th ed.). Pearson.